

BACKEND INTERVIEW ROADMAP

System Design Questions with Answers



Master these concepts step by step. Each question has a short, clear explanation.

1. AUTHENTICATION & SECURITY



Focus: Secure access, tokens, passwords, sessions & protection.

Q1 If JWT is stateless, how does “Logout from All Devices” work?

Answer: We keep a record of issued tokens (or token IDs) in a database/Redis. On logout, we invalidate all tokens for that user by blacklisting them. When a request comes, we check the token against the blacklist.

Q2 A hacker stole a JWT token. How would you minimize the impact?

Answer: Use short-lived access tokens (5-15 mins) and long-lived refresh tokens. Store refresh tokens securely in DB and allow user to revoke sessions. Enable device tracking and force logout suspicious sessions.

Q3 How would you design authentication for millions of users?

Answer: Use a scalable auth service with load balancers. Store user data in a database (sharded if needed). Use JWT for stateless auth and Redis for session/blacklist storage.

2. DATABASES



Focus: Store data efficiently, scale it and keep it safe.

Q1 What's the difference between Sharding and Partitioning?

Answer: Partitioning splits a table into parts within the same database (vertical/horizontal). Sharding splits data across multiple databases/servers. Sharding helps scale out; Partitioning helps manage large tables.

Q2 Why is scaling writes harder than scaling reads?

Answer: Reads can be served from replicas and caches. Writes require consistency and updates to the main database, which becomes a bottleneck as traffic grows.

Q3 How would you handle database failover?

Answer: Use a primary-replica setup. If primary fails, a replica is promoted to primary. Use automated failover with health checks and monitoring.

3. CACHE & REDIS



Focus: Speed up systems, reduce database load.

Q1 Why use Redis if we already have a database?

Answer: Databases are disk-based and slower. Redis is in-memory and ultra-fast. We use Redis for caching, sessions, rate limiting, leaderboards, etc.

Q2 What is a Cache Stampede and how would you prevent it?

Answer: When many requests try to load the same missing key and hit the database at the same time. Prevent it using techniques like: TTL with jitter, mutex locks, and request coalescing.

Q3 How would you invalidate cache in a large-scale system?

Answer: Use strategies like: TTL (time-based expiry), event-based invalidation, versioning keys, or write-through / write-behind patterns.

Pro Tip: Always think in terms of scale, reliability, performance and user experience.



Understand the Problem



Design Step by Step



Discuss Trade-offs & Alternatives



Calculate Scale & Constraints



Wrap up with Key Takeaways

BACKEND INTERVIEW ROADMAP - PAGE 2

System Design Questions with Answers



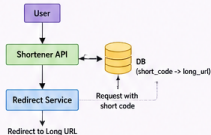
4. APIs & MICROSERVICES



Focus: Build reliable, scalable and loosely coupled services.

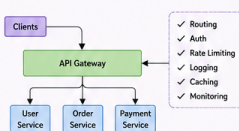
Q1 How would you design a URL Shortener like Bitly?

Answer: Use a write API to create short URLs and store mapping in a database (e.g., short_code -> long_url). Use a Base62 encoding for short codes. For high scale: use Redis for caching and a distributed counter or Snowflake ID generator.



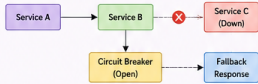
Q2 What is an API Gateway and why is it used?

Answer: API Gateway is a single entry point for all client requests. It handles routing, authentication, rate limiting, logging, caching and monitoring. It hides internal services and reduces client complexity.



Q3 One microservice is down. How do you prevent cascading failures?

Answer: Use timeouts, retries with exponential backoff, and circuit breakers. Implement bulkheads (resource isolation) and fallbacks to keep the system stable.



5. MESSAGING & QUEUES



Focus: Decouple services and handle work asynchronously.

Q1 How does WhatsApp deliver messages to offline users?

Answer: When a user is offline, the message is stored in a persistent queue. Once the user comes online, the message is delivered. This is typically implemented using distributed message queues like Kafka or RabbitMQ.



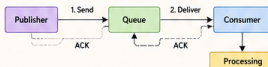
Q2 When would you choose Kafka over RabbitMQ?

Answer: Kafka is better for high-throughput, event streaming, and log aggregation. RabbitMQ is better for task queues, complicated routing and reliable message delivery.

	Kafka	RabbitMQ
High Throughput	Yes	Moderate
Message Persistence	Yes (Log based)	Yes
Ordering	Per Partition	Per Queue
Use Case	Event Streaming	Task Queues

Q3 How would you guarantee message delivery?

Answer: Use acknowledgements (ACKs). Publisher confirms the message is sent. Consumer ACKs after processing. Use retries and dead-letter queues for failed messages.



6. SCALABILITY



Focus: Handle more users, more traffic and more data.

Q1 One API endpoint receives 1 million requests per minute. How would you scale it?

Answer: Use load balancers to distribute traffic. Scale horizontally (add more servers). Use caching (Redis/CDN) to reduce database load. Use database read replicas and optimize queries.



Q2 How would you handle traffic spikes during a flash sale?

Answer: Use autoscaling to add more servers. Use caching for product data. Use queues for order processing. Implement rate limiting and circuit breakers.



Q3 How would you design a highly available system?

Answer: Deploy across multiple availability zones. Use load balancers, health checks and auto-failover. Keep no single point of failure. Backups and monitoring are critical.



KEY TAKEAWAYS



Understand the problem deeply



Design simple, scalable solutions



Think about failure, security & reliability



Optimize for scale, performance & cost



Keep learning and build real projects

Q1 The payment succeeded but the order failed. What happens next?

Flow:



02 How would you prevent duplicate payments?

```
graph LR; Client[Client] -- "Payment Request (Idempotency Key)" --> PaymentService[Payment Service]; PaymentService --> IdempotencyStore[(Idempotency Store (DB/Redis))]; IdempotencyStore -- "Return Previous Result" --> Client;
```

The diagram shows a Client sending a Payment Request (Idempotency Key) to a Payment Service. The Payment Service then interacts with an Idempotency Store (DB/Redis). The Idempotency Store returns the previous result to the Client.

03 Why do refunds take longer than payments?

Customer ← Bank ← Payment Gateway ← Merchant Account

Refund Flow (Slower)

Q1 How does YouTube count views without updating the database every second?

```

graph LR
    A[High Volume View Events] --> B[Event Queue  
(Kafka/PubSub)]
    B --> C[Aggregation Service]
    C --> D[(Analytics DB)]
    E((Clock)) -- "Aggregate every  
few seconds/minutes" --> C
  
```

02 How would Uber find the nearest driver?

Answer: Store driver locations in a geospatial index (e.g., Redis GEO, PostGIS). When a user requests a ride, query nearby drivers using geo-radius search and match based on availability and ratings.



03 How would Instagram generate a personalized feed?

Answer: Use a ranking system.
Collect signals (likes, follows, recency, relationship).
Feed candidates are fetched from multiple sources (following, popular, explore) and ranked by ML/model.
Top N items are returned to the user.



01 How would you perform a deployment with zero downtime?

Answer: Use Blue-Green or Rolling Deployment.
New version is deployed alongside the old version.
Traffic is gradually shifted to the new version.
If everything is healthy, old version is retired.



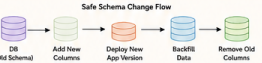
02 How would you roll back a failed deployment?

Answer: Keep the previous stable version available.
If health checks fail, automatically route traffic back to the old version (auto-rollback).
Investigate logs and fix the issue.



Q3 How would you deploy database schema changes safely?

Answer: Use backward compatible changes.
Add new columns (nullable) first.
Deploy code that can work with both old and new schemas.
Backfill data if needed. Then remove old columns.



01 How would you generate unique IDs across multiple servers?

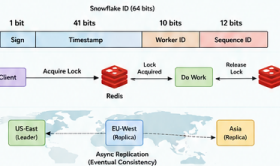
Answer: Use Snowflake Algorithm (e.g., Twitter Snowflake). It uses timestamp + server ID + sequence number to generate unique, k-sortable IDs.

Q2 How would you implement a distributed lock?

Answer: Use Redis SET key value NX PX <expiry>.
If lock acquired, the process owns the lock.
Release lock by deleting the key (only if you own it).
Helps in preventing race conditions.

03 How would you handle data consistency across regions?

Answer: Use eventual consistency for most features. For critical data, use strong consistency with leader election and synchronous replication. Use techniques like versioning, timestamps, and conflict resolution.



 Remember: There is no single “perfect” design. Communicate, make assumptions, explain trade-offs, and show that you can scale and iterate.



Clarify Requirements



Discuss
Trade-offs



- Think Scalable



Keep it Simple